PROJECT DOCUMENTATION

Implement Client Script & UI Policy (Incident)

Team members

- L.POOJA - TEAM LEAD
- M.SHAKTHIPRIYA
- K.PUNITHA

TABLE OF CONTENTS

1. Introduction

2. Ideation Phase

   2.1 Problem Statement

   2.2 Empathy Map Canvas

   2.3 Brainstorming

3. Requirement Analysis

   3.1 Customer Journey Map

   3.2 Solution Requirements

   3.3 Data Flow Diagram

   3.4 Technology Stack

4. Project Design

   4.1 Problem-Solution Fit

   4.2 Proposed Solution

   4.3 Solution Architecture

   4.4 Configuration Summary

5. Project Planning & Scheduling

6. Functional and Performance Testing

7. Results

8. Advantages & Disadvantages

9. Conclusion

10. Future Scope

11. Appendix

INTRODUCTION

1.1 Project Overview

The project titled “Implement Client Script & UI Policy (Incident)” focuses on improving the accuracy and consistency of Incident records through client-side controls and validation.

The implementation uses UI Policies and Client Scripts to dynamically control Incident form behavior based on the value of the Impact field. When an Incident has High Impact, specific controls are automatically applied to ensure that important information is entered correctly.

The project implements a UI Policy named “High Impact Control”, an onChange Client Script for automatically setting Urgency, an onSubmit Client Script for validating the Assigned To field, and an onCellEdit Client Script for preventing direct State changes through list editing.

1.2 Purpose

The purpose of this project is to ensure that Incident records follow defined business rules before they are submitted or modified.

- Make Assigned To mandatory for High Impact Incidents.
- Make Urgency read-only when Impact is High.
- Automatically set Urgency to High when Impact is High.
- Prevent an Incident from being saved when Assigned To is missing for a High Impact Incident.
- Prevent State changes through direct list editing.
- Allow State changes through the Incident form.

2. IDEATION PHASE

2.1 Problem Statement

Incident records require consistent and accurate data entry for effective triage, routing, and resolution. However, relying only on user awareness and manual checks can result in incomplete, inconsistent, or incorrect information.

For example, a High Impact Incident may be submitted without an Assigned To value, or users may directly modify the State field through list editing without following the intended form-based process.

2.2 Empathy Map Canvas

Area	Project Context
Says	I need to create and update Incident records
Think	Important Incident information should not be missed.
Does	Creates Incidents and enters Impact, Assignment Group, Assigned To, Urgency and State.
Feels	Needs clear and controlled form behavior
Pain Points	Missing required information, inconsistent values and unwanted list-edit changes.
Needs	Automatic field behavior, validation and controlled updates.

2.3 Brainstorming

The following ideas were considered to improve Incident data entry and control:

- Make Assigned To mandatory for High Impact Incidents.
- Automatically set Urgency when Impact is High.
- Make Urgency read-only for High Impact Incidents.
- Prevent saving an Incident when required information is missing.
- Prevent direct State modification through list editing.
- Allow State changes through the Incident form.
- Reverse field restrictions when the Incident no longer meets the specified condition.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	System Behavior
1. Create Incident	Opens a new Incident	Incident form is displayed
2. Set Impact	Selects High Impact	UI Policy and onChange Client Script ar
3. Assignment	Leaves Assigned To empty	Assigned To becomes mandatory
4. Urgency	Impact is High	Urgency is automatically set to High and
5. Submit	Attempts to save without Assigned To	onSubmit Client Script prevents submis
6. Complete Assignment	Selects an Assigned To user	Validation condition is satisfied
7. Submit	Saves the Incident	Incident is successfully saved
8. List Editing	Attempts to edit State directly from list	onCellEdit Client Script blocks the chan
9. Form Editing	Changes State through Incident form	State change is allowed

3.2 Solution Requirements

Functional Requirements

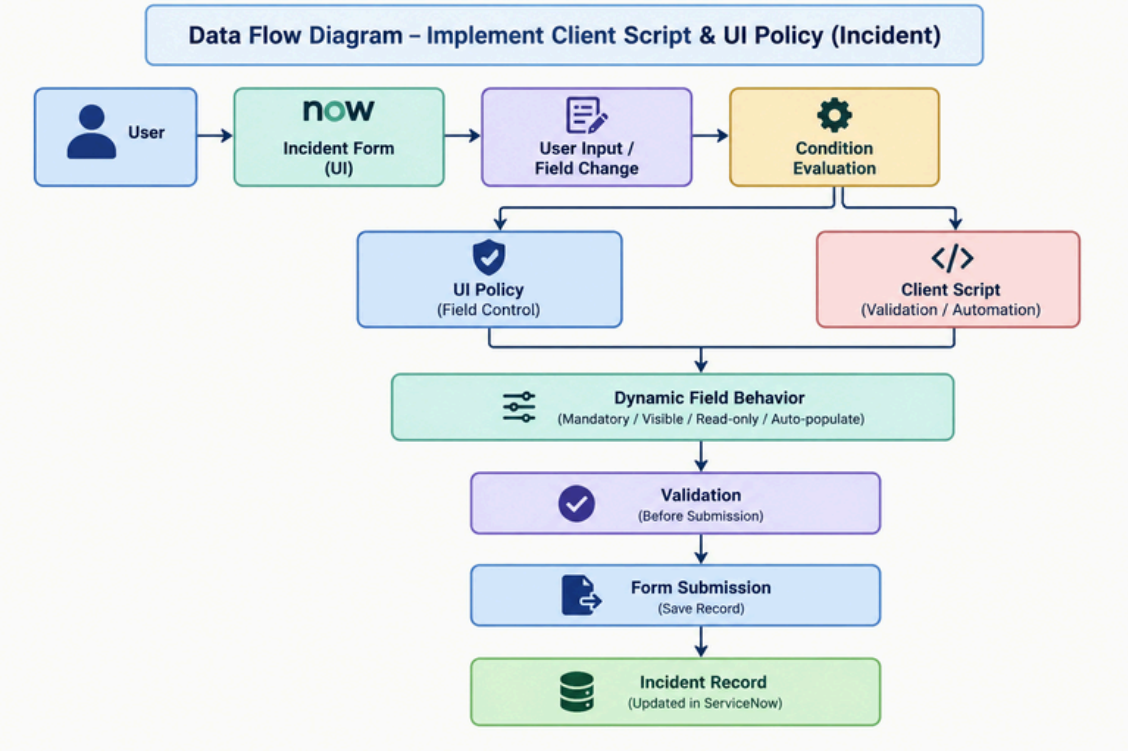
The system shall:

1. Apply the High Impact Control UI Policy to Incident records.
2. Trigger the UI Policy when Impact = 1 – High.
3. Make Assigned To mandatory for High Impact Incidents.
4. Make Urgency read-only for High Impact Incidents.
5. Automatically set Urgency to High when Impact is High.
6. Prevent saving a High Impact Incident when Assigned To is empty.
7. Display an appropriate validation message when submission is prevented.
8. Prevent State changes through direct list editing.
9. Allow State changes through the Incident form.
10. Reverse UI Policy restrictions when the Impact condition becomes false.

Non-Functional Requirements

1. The implementation should be lightweight.
2. The system should respond immediately to user actions.
3. The solution should provide consistent behavior.

3.3 Data Flow Diagram



3.4 Technology Stack

Technology	Purpose
ServiceNow	Development and implementation platform
Incident Management	Module used for the project
UI Policy	Controls field behavior based on conditions
UI Policy Action	Defines field-level behavior
Client Script	Implements client-side logic and validation
JavaScript	Programming language used for Client Scripts
Web Browser	Used to access and test the application
Ust Editing	Incident is succesyfully saved
Form Editing	Changes impact from rligh to Medium

4. PROJECT DESIGN

4.1 Problem-Solution Fit

The project addresses the problem of inconsistent Incident data entry an uncontrolled field modification.

The solution uses conditional controls based on the Impact field.

When Impact is High:

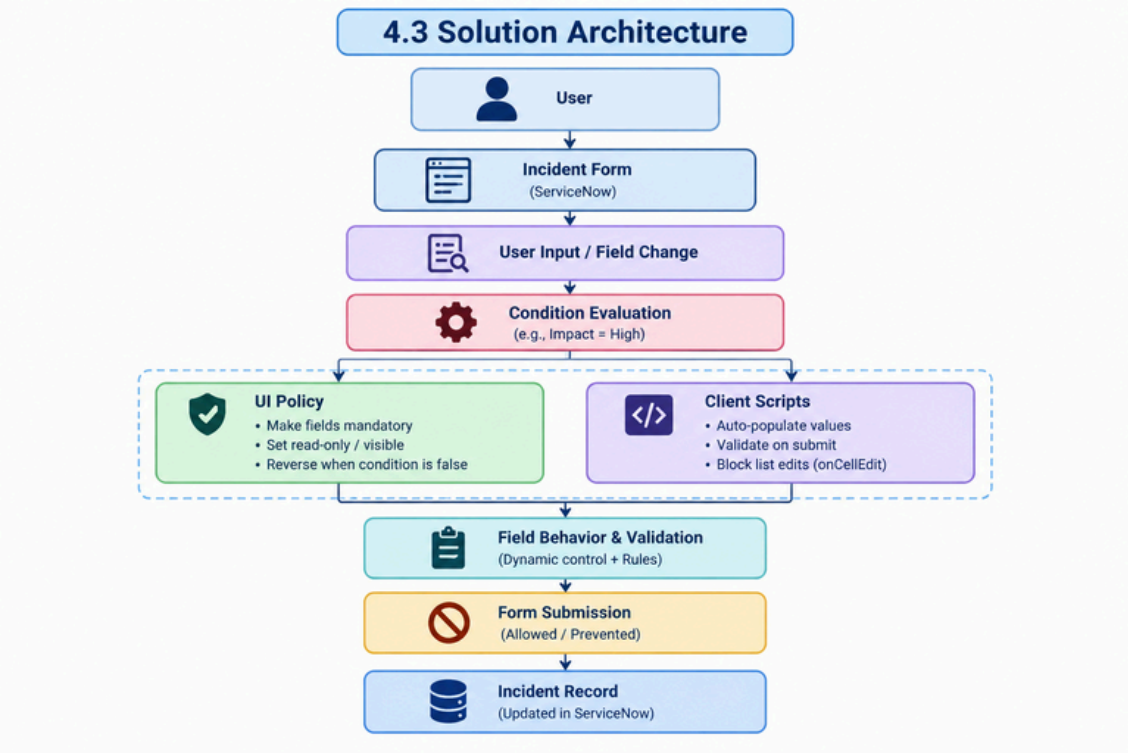
- Assigned To becomes mandatory.
- Urgency becomes read-only.
- Urgency is automatically set to High.
- Saving is prevented if Assigned To is empty.
- The solution also controls State modification by preventing direct list editing while allowing State changes through the Incident form.
- This ensures that Incident records follow the required business rules while providing users with appropriate feedback.

4.2 Proposed Solution

The proposed solution uses ServiceNow UI Policies and Client Scripts to control Incident form fields dynamically. It makes fields mandatory, read-only, or automatically updated based on conditions, while preventing invalid submissions and improving data accuracy.

4.3 Solution Architecture

The solution architecture consists of the user interacting with the ServiceNow Incident Form, where inputs are evaluated based on predefined conditions. UI Policies and Client Scripts then control field behavior, perform validation, and automate actions. After validation, the form is either submitted or prevented, resulting in an updated Incident record.



The architecture combines multiple client-side controls to provide complete validation and field management.

5. PROJECT PLANNING & SCHEDULING

Phase	Activity
Phase 1	Understand ServiceNow Incident Management
Phase 2	Identify the problem and requirements
Phase 3	Study Client Scripts
Phase 4	Study UI Policies
Phase 5	Create UI Policy
Phase 6	Create Client Script
Phase 7	Configure and test the Incident form
Phase 8	Perform functional and performance testing
Phase 9	Capture output screenshots
Phase 10	Prepare project documentation

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

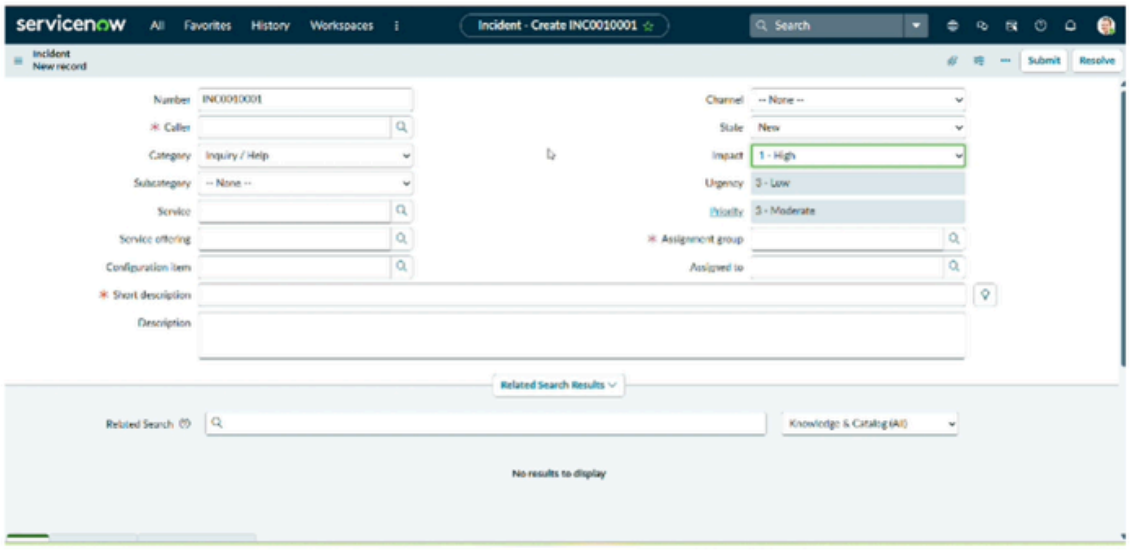
Testing was performed to verify whether the Client Script and UI Policy work correctly on the Incident form.

Test Case 1 – Mandatory Enforcement

Condition: Impact = High and Assigned To is empty.

Expected Result:

- Assigned To should be mandatory.
- Incident should not be saved.
- An error message should be displayed.



Test Case 2 – Successful Save

Condition: Impact = High and Assigned To contains a valid user.

Expected Result:

- Incident should save successfully.
- Urgency should be automatically set to High.
- Urgency should remain read-only.

Result: Incident saves successfully after the required information is provided.

Test Case 3 – Reverse Condition

Condition: Impact is changed from High to Medium.

Expected Result:

1. Assigned To should no longer be mandatory.
2. Urgency should become editable.
3. UI Policy restrictions should be reversed.

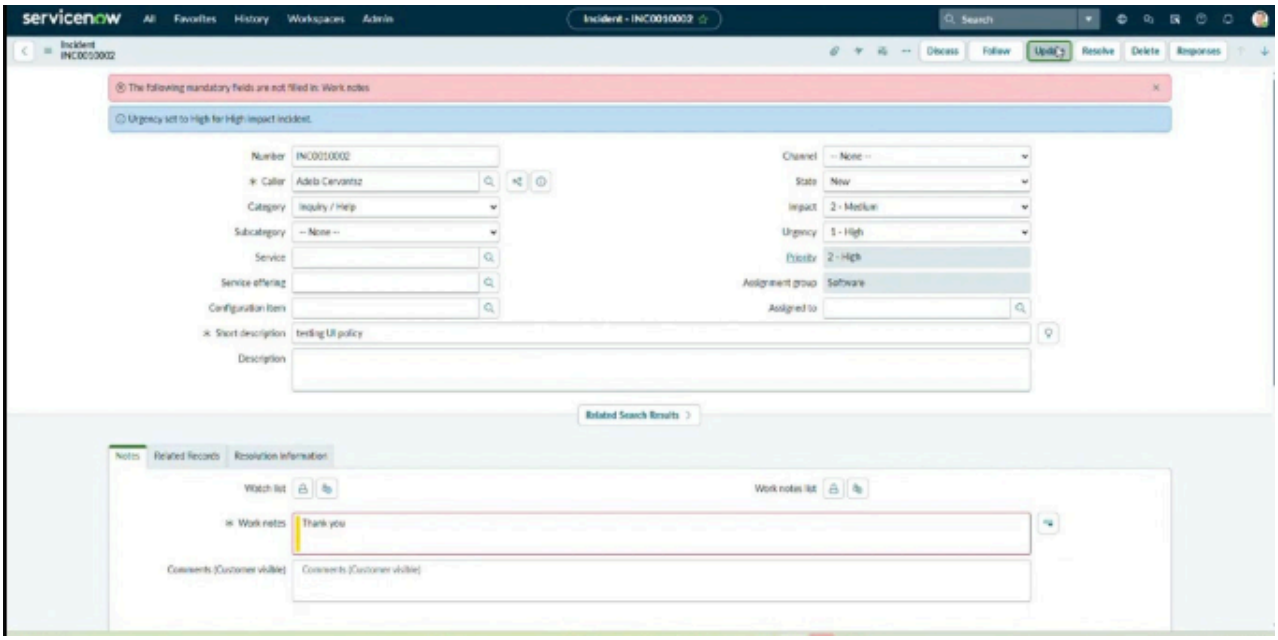
Result: The configured restrictions are reversed when the condition is no longer satisfied.

Test Case 4 – List Edit Blocking

Condition: User attempts to change State directly from the Incident list.

Expected Result:

- Alert message should appear.
- State should remain unchanged.
- Result: Direct list editing of State is blocked.



Test Case 5 – Form-Based State Update

Condition: User changes State from the Incident form.

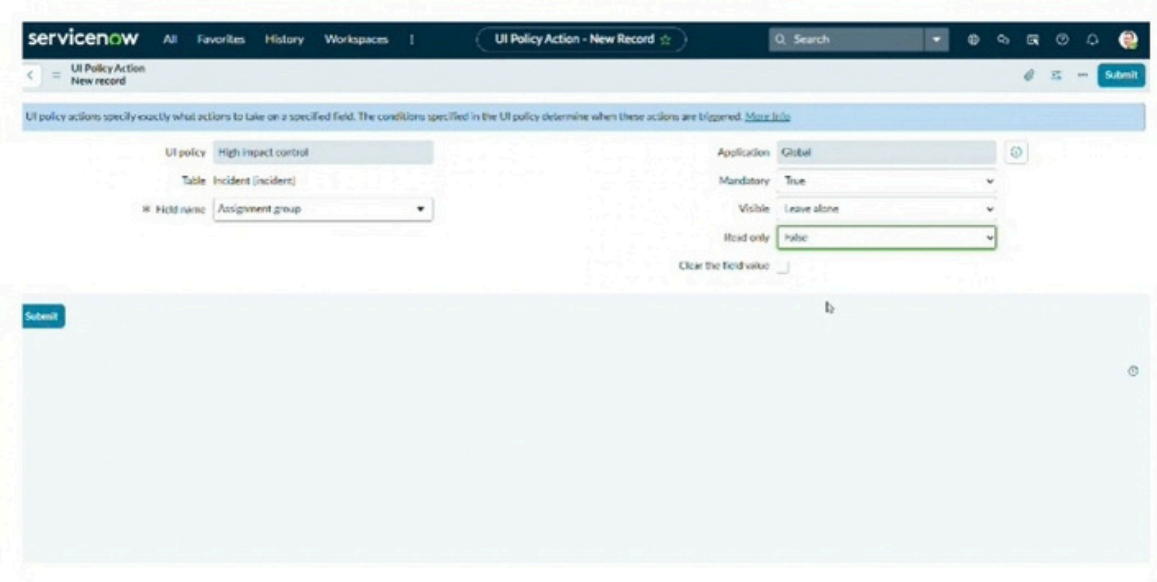
Expected Result:

- State change should be allowed.
- Incident should save successfully.
- Result: State can be updated through the Incident form.

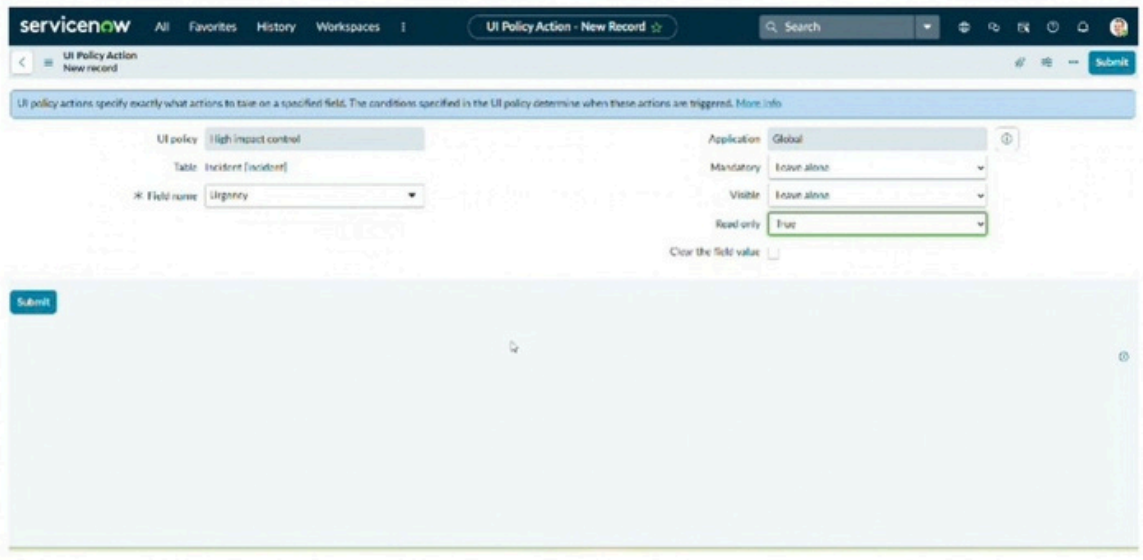
7. RESULTS

Result Evidence

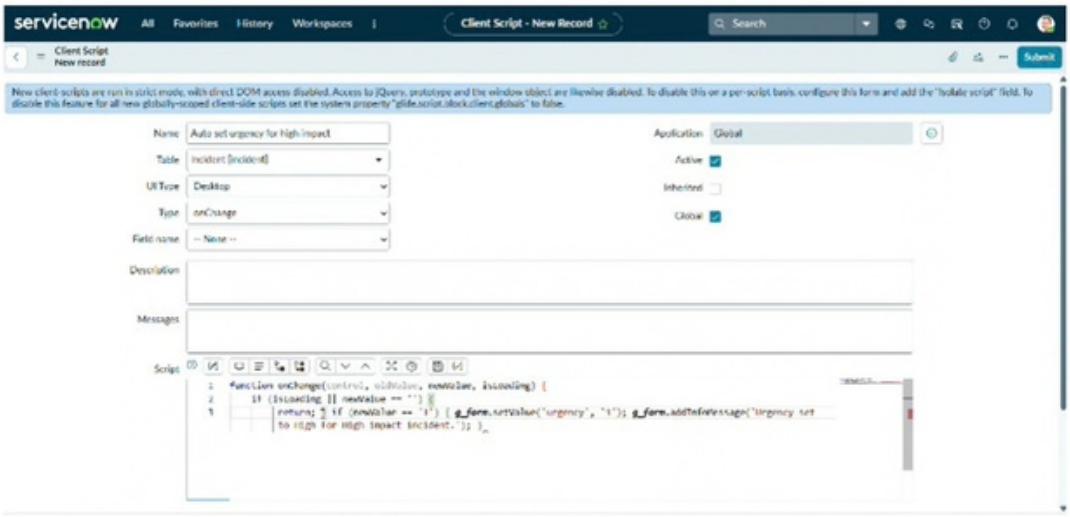
1. High Impact Control UI Policy



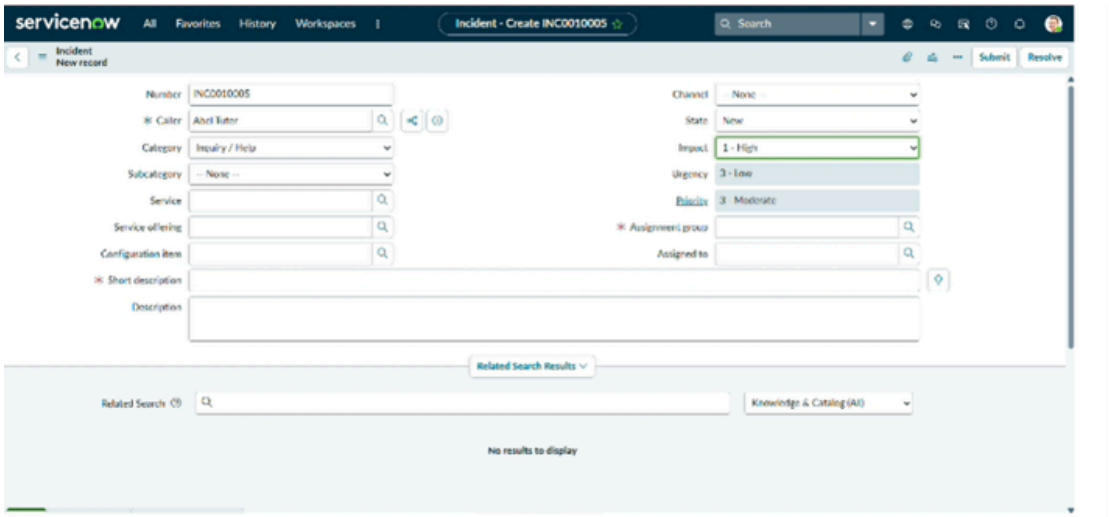
2.UI Policy Action



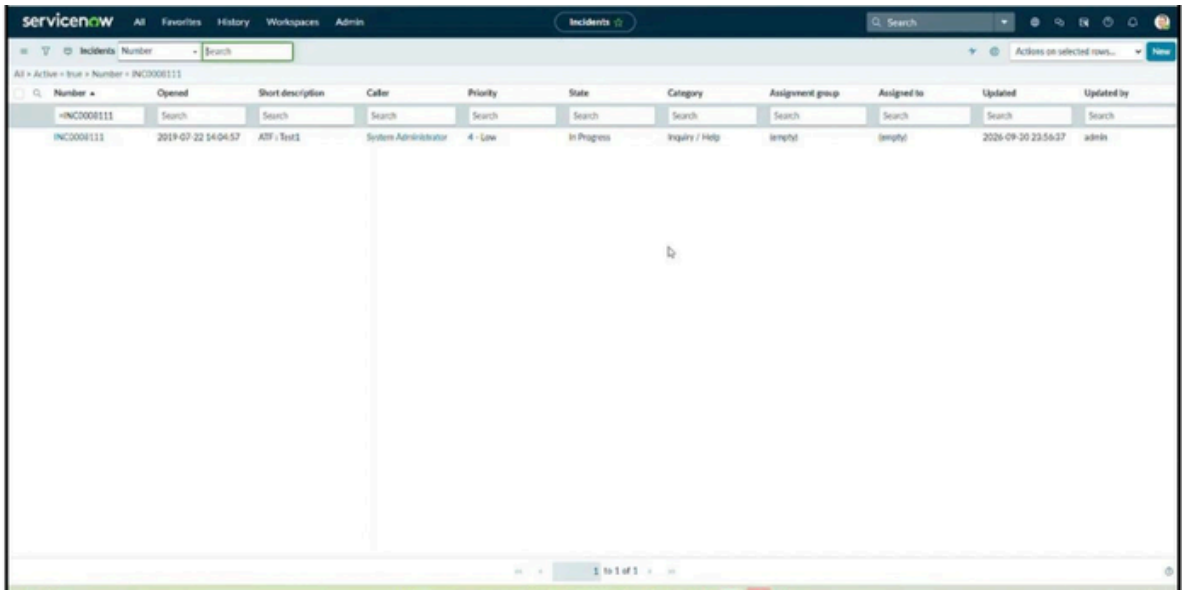
4.onChange Client Script



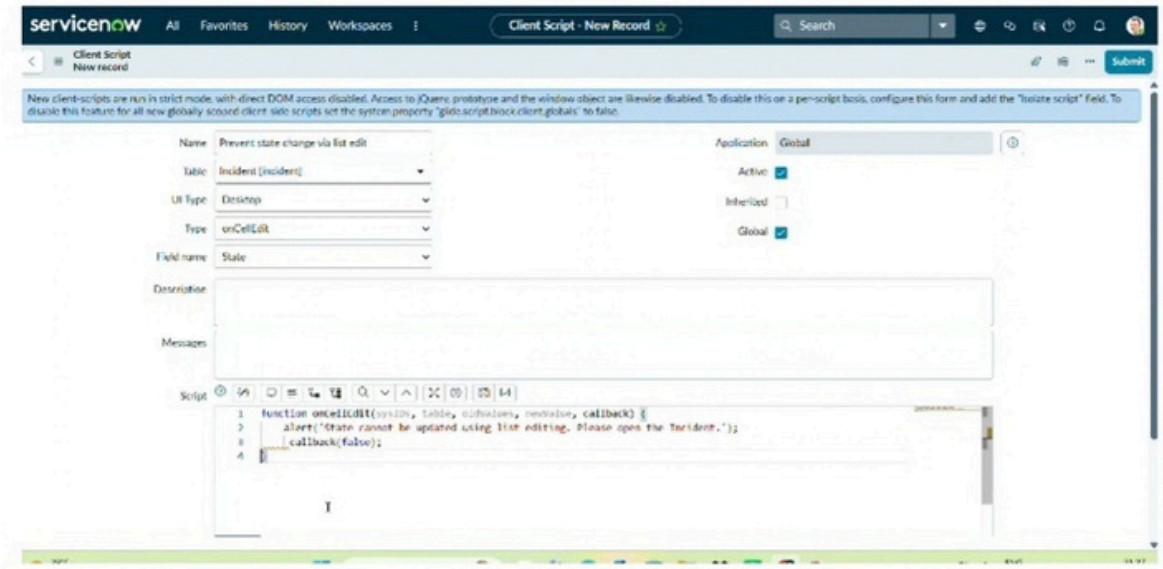
5.Impact changed from high to medium



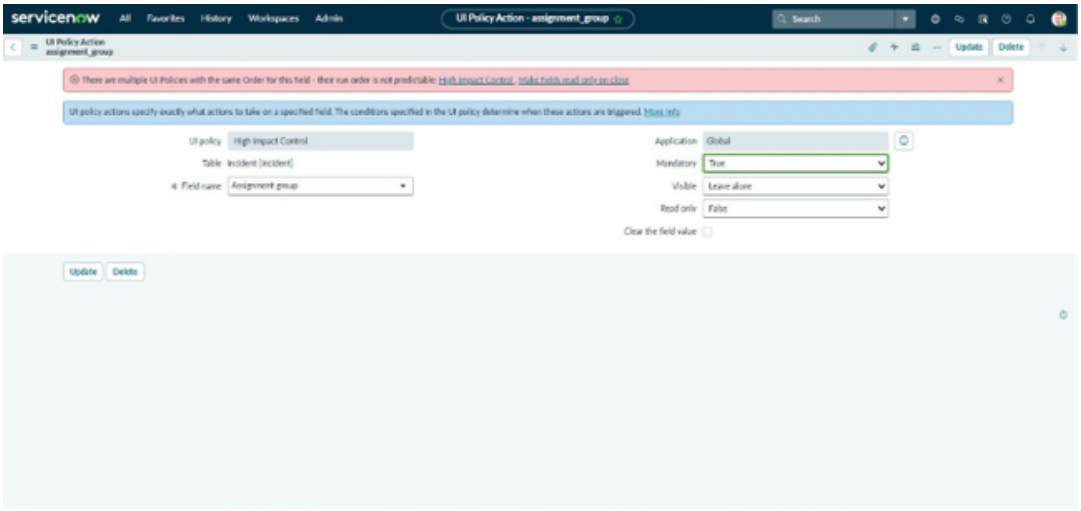
5.onSubmit Validation



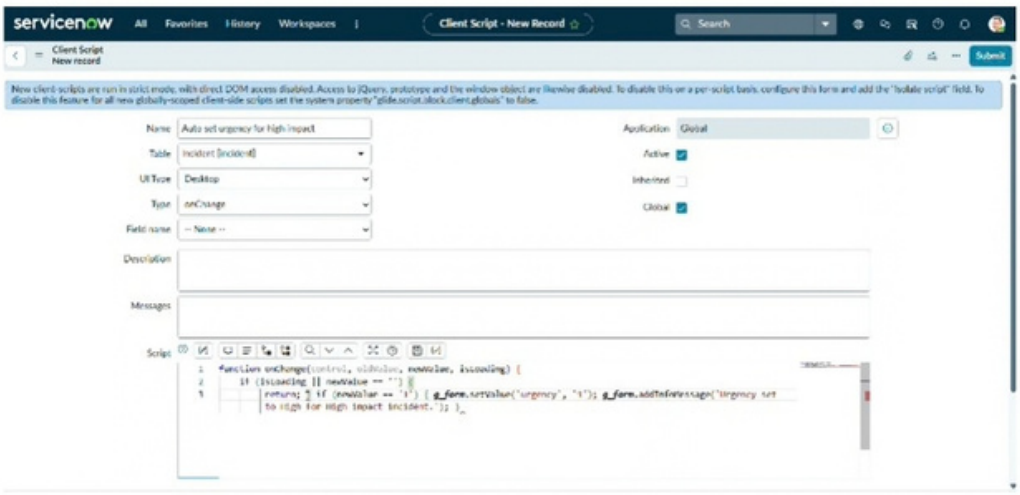
6.onCellEdit Client Script



7.Assignment Group Mandatory UI Policy Action



8.Onchange client script



ADVANTAGES & DISADVANTAGES

Advantages

- Improves Incident data integrity.
- Ensures required information is provided.
- Automatically sets Urgency for High Impact Incidents.
- Prevents incorrect Incident submission.
- Controls unauthorized State changes through list editing.
- Provides immediate feedback to users.
- Reduces manual validation.
- Improves consistency of Incident records.
- Provides dynamic field behavior.

Disadvantages

- Incorrect configuration may result in unexpected field behavior.
- Complex Client Scripts may require maintenance.
- Multiple client-side controls can increase configuration complexity.
- Changes in business requirements may require script modifications.
- The solution depends on correctly configured conditions and field values.

9. CONCLUSION

- The Implement Client Script & UI Policy (Incident) project successfully demonstrates the use of client-side controls to improve Incident data integrity and form usability.
- The High Impact Control UI Policy dynamically controls the Assigned To and Urgency fields when Impact is High. The onChange Client Script automatically sets Urgency to High, while the onSubmit Client Script prevents the submission of a High Impact Incident when Assigned To is missing.
- The onCellEdit Client Script further improves control by preventing direct State changes through list editing while allowing updates through the Incident form.
- Overall, the implementation provides conditional field control, automatic value updates, save-time validation, and controlled user interaction, resulting in more consistent and reliable Incident records.

10. FUTURE SCOPE

The project can be extended with additional automation and validation features.

Future enhancements may include:

- Implementing additional UI Policies for different Incident conditions.
- Adding more Client Scripts for field validation.
- Automating Assignment Group selection.
- Automatically assigning Incidents based on Category and Subcategory.
- Adding notifications for High Impact Incidents.

11. APPENDIX

Source Code:

11.1 onChange Client Script – Auto Set Urgency

```
function onChange(control, oldValue, newValue, isLoading) {
if (isLoading || newValue == '') {
return;
}

if (newValue == '1') {
g_form.setValue('urgency', '1');
g_form.addInfoMessage(
'Urgency set to High for High impact incident.'
);
}
}
```

11.2 onSubmit Client Script – Assigned To Validation

```
function onSubmit() {
if (g_form.getValue('impact') == '1' &&
g_form.getValue('assigned_to') == '') {

g_form.showErrorBox(
'assigned_to',
'Assigned To is mandatory for High impact incidents.'
);

return false;
}

return true;
}
```

11.3 onCellEdit Client Script – State Change Restriction

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

alert(
'State cannot be updated using list editing. Please open the Incident.'
);

callback(false);
}
```